

Chapter 2: Heuristics for the Vehicle Routing Problem

Nature Inspired Optimisation for Delivery Problems (2022)

Lecture Slides

Chapter Overview I

This chapter introduces the **Vehicle Routing Problem (VRP)** — one of the most practically important combinatorial optimisation problems, arising whenever goods must be delivered from a central warehouse to a dispersed set of customers. Unlike the Travelling Salesman Problem (TSP) studied in Chapter 1, the VRP requires planning *multiple* routes simultaneously, subject to the constraint that each vehicle can carry only a limited load.

The chapter is organised as follows:

- **Section 2.1** defines the Capacitated Vehicle Routing Problem (CVRP) precisely, introducing the concepts of routes, demands, and vehicle capacity.
- **Section 2.2** introduces three algorithms for solving the CVRP: the *Grand Tour* construction, the *Clarke–Wright savings* algorithm, and discusses the 2-opt local-search improvement.
- **Section 2.3** presents a Java implementation (with Python equivalents) and describes the class structure.
- **Section 2.4** benchmarks both heuristics on the standard Augerat problem sets and compares them with branch-and-cut.

- **Section 2.5** draws conclusions and motivates the need for more powerful metaheuristics.

Key Takeaway

Simple heuristics such as Clarke–Wright are fast and easy to implement, but produce solutions that are 5–27% longer than optimal. More sophisticated nature-inspired techniques (introduced in later chapters) are needed to close this gap.

Motivation: From TSP to VRP I

The **Travelling Salesman Problem (TSP)** asks for the shortest *single* tour that visits every city exactly once. In reality, however, a delivery company rarely has just one vehicle. Instead, it operates a *fleet*, and each vehicle has a maximum weight or volume it can carry. The question then becomes: how should we divide customers among vehicles, and in what order should each vehicle visit its assigned customers?

This is the **Vehicle Routing Problem (VRP)**, first studied by Dantzig and Ramser in 1959. The VRP generalises the TSP: every VRP solution consists of a set of TSP-like tours (called *routes*), each starting and ending at a central **depot**.

Motivation: From TSP to VRP II

The Capacitated VRP (CVRP) — Informal Statement

Given a depot and a set of customers, each with a known demand, assign customers to vehicles and plan a route for each vehicle such that:

- 1 every customer is visited exactly once,
- 2 the total demand on any route does not exceed the vehicle capacity Q (Q , the maximum load a vehicle can carry),
- 3 the number of vehicles used is minimised,
- 4 the total distance driven by all vehicles is minimised.

The two objectives (minimise vehicles, minimise distance) are often treated lexicographically: first minimise vehicles, then minimise distance.

Motivation: From TSP to VRP III

Running Example: ACE Doughnuts

Throughout this chapter we use a concrete motivating scenario. ACE Doughnuts operates a factory that bakes doughnuts to order. Customers place orders the previous day. The factory owns a fleet of vans, each capable of carrying *capacity* doughnuts. Each day a delivery plan must be computed satisfying:

- No van carries more than *capacity* doughnuts.
- Every customer receives exactly the number they ordered.
- The number of vans used is minimised.
- The total distance driven by all vans is minimised.

This scenario maps directly onto the CVRP: customers correspond to delivery addresses, demands correspond to order sizes, the depot is the factory, and vehicle capacity is the maximum load per van.

Formal Definition of the CVRP I

We now define the CVRP precisely. The problem is specified by a graph whose nodes represent the depot and the customers.

Notation

n	number of customers
d	depot (the start/end location of all routes)
c_i	customer i , for $i = 1, \dots, n$
δ_i	demand (delta, δ) of customer i : the quantity that must be delivered to c_i
Q	vehicle capacity: the maximum total demand a single vehicle can service on one route
$\text{dist}(a, b)$	Euclidean distance between nodes a and b

Here δ_i (delta) is the Greek letter commonly used for “difference” or “quantity”; in this context it represents how many units customer i needs. The symbol Q (for Quantity/Capacity) is the hard upper limit on how much any single vehicle can carry.

Formal Definition of the CVRP II

A CVRP Solution

A **solution** is a set of routes $R = \{r_1, r_2, \dots, r_k\}$, where k is the number of vehicles. Each route r_j is an ordered list of customers:

$$r_j = (d, c_{j_1}, c_{j_2}, \dots, c_{j_{m_j}}, d)$$

meaning the vehicle departs from the depot d , visits customers $c_{j_1}, c_{j_2}, \dots$ in that order, and returns to d .

Formal Definition of the CVRP III

Feasibility Constraints

A solution is **feasible** if and only if:

- ① **Coverage**: every customer c_i appears in exactly one route.
- ② **Capacity**: for every route r_j ,

$$\sum_{i \in r_j} \delta_i \leq Q$$

That is: the sum (Σ , sigma, meaning “add up”) of demands of all customers in route r_j does not exceed the vehicle capacity Q .

The capacity constraint is the key feature distinguishing CVRP from TSP. Think of it as a weight limit: once a van is full, it cannot take on more parcels — it must return to the depot first.

Formal Definition of the CVRP IV

Objective Function

The total cost of a solution (total distance driven by all vehicles) is:

$$\text{TotalDist}(R) = \sum_{j=1}^k \text{dist}(r_j)$$

where $\text{dist}(r_j)$ is the total distance of route r_j , computed as the sum of consecutive edge lengths:

$$\text{dist}(r_j) = \text{dist}(d, c_{j_1}) + \sum_{l=1}^{m_j-1} \text{dist}(c_{j_l}, c_{j_{l+1}}) + \text{dist}(c_{j_{m_j}}, d)$$

Formal Definition of the CVRP V

How to Read This Formula

The outer sum adds up the distance of each individual route. For a single route, the distance is: depot to first customer, then customer to customer along the route, then last customer back to depot. Doubling the number of customers on a route roughly doubles the distance driven for that route — but the capacity Q acts as a ceiling that forces the route to be split before it grows too long.

Formal Definition of the CVRP VI

Minimum Number of Vehicles

A lower bound on the number of vehicles required is:

$$k_{\min} = \left\lceil \frac{\sum_{i=1}^n \delta_i}{Q} \right\rceil$$

Here $\lceil \cdot \rceil$ (ceiling function) rounds up to the nearest integer, because we cannot use a fractional vehicle.
 $\sum_{i=1}^n \delta_i$ is the total demand across all customers.

Formal Definition of the CVRP VII

Worked Example: 8-Customer Instance

The book's example has 8 customers with demands

$\delta_1 = 5$, $\delta_2 = 4$, $\delta_3 = 6$, $\delta_4 = 5$, $\delta_5 = 2$, $\delta_6 = 3$, $\delta_7 = 3$, $\delta_8 = 4$ and vehicle capacity $Q = 10$.

Total demand = $5 + 4 + 6 + 5 + 2 + 3 + 3 + 4 = 32$.

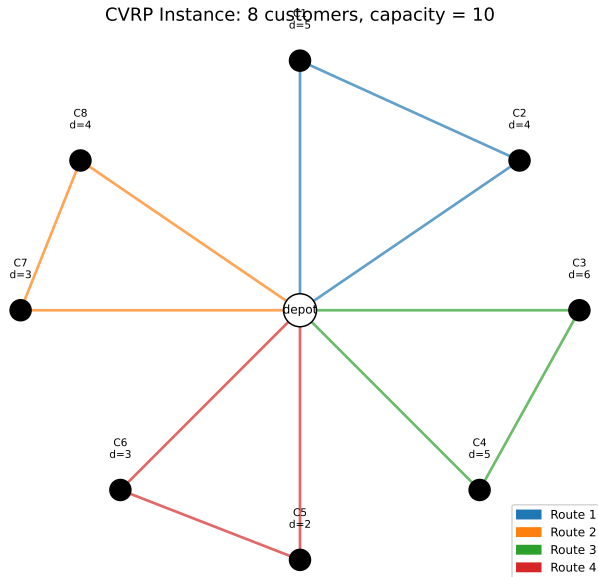
$k_{\min} = \lceil 32/10 \rceil = \lceil 3.2 \rceil = 4$ vehicles.

So at least 4 vehicles are needed. The solution shown in the figure below achieves exactly 4 routes and is therefore optimal in terms of vehicle count.

CVRP Example: 8 Customers, 4 Routes I

The figure below shows a valid solution to the small CVRP instance. The depot is at the centre. Each customer node is labelled with its demand d . Four coloured routes depart from and return to the depot, each respecting the capacity constraint $Q = 10$.

CVRP Example: 8 Customers, 4 Routes II



Benchmark Problem Instances: Augerat Sets I

To evaluate algorithm performance, researchers use *benchmark instances* — problem instances whose optimal (or best-known) solution is already known, so that any new algorithm can be compared against it fairly.

This chapter uses the **Augerat (1995) benchmark**, a widely-used collection of 60 CVRP instances grouped into three sets:

- Set A** Random instances. Customers are placed at uniformly random positions over a square region.
- Set B** Clustered instances. Customers tend to group together in geographic clusters, which often makes routing easier.
- Set P** Literature-based instances. These come from earlier published VRP research and have varied structure.

The number of customer visits n ranges from 15 to 100, with an average of 51. Some instances have a known *optimal* solution (found by branch and cut); others only have a *best-known* solution.

Benchmark Problem Instances: Augerat Sets II

The figure below shows one representative instance from each set. The red square is the depot; black dots are customer locations.



Fig. 2.2 Examples of problem instances from each of the Augerat sets (Left to right: A-n32-K5, B-n57-k9 and P-n76-k4)

Benchmark Problem Instances: Augerat Sets III

Visually, Set A instances look like scattered points (no structure), while Set B instances show obvious clusters. Set P instances have mixed structure inherited from their original authors.

Why Benchmarks Matter

Without a common benchmark, it is impossible to compare two algorithms objectively. The Augerat set provides the “ground truth” against which the Grand Tour and Clarke–Wright solvers are evaluated in Section 2.4.

Overview of Solution Approaches I

The CVRP is **NP-hard**: no polynomial-time algorithm is known to find the optimal solution in general. For practical problem sizes (tens to hundreds of customers), we therefore turn to **heuristics** — algorithms that find good (but not necessarily optimal) solutions in reasonable time.

This section covers two construction heuristics:

- 1 The **Grand Tour** heuristic: build a TSP tour over all customers first, then split it into capacity-feasible sub-tours.
- 2 The **Clarke–Wright Savings Algorithm**: start with the worst possible solution (one vehicle per customer) and iteratively merge routes whenever doing so saves distance.

Both produce *feasible* solutions quickly, but with different quality. Neither guarantees optimality.

The Grand Tour Heuristic I

The Grand Tour approach is conceptually the simplest way to build a CVRP solution. The key idea is: *first ignore the capacity constraint and solve an easier problem (the TSP), then enforce the constraint by splitting the single big tour into smaller feasible routes.*

Step 1 — Solve a TSP. Treat all customers and the depot as cities in a TSP. Use any TSP solver (for example, the Nearest-Neighbour heuristic from Chapter 1) to find a tour that visits every customer once and returns to the depot. This produces a single, potentially very long route:

$$d \rightarrow c_1 \rightarrow c_2 \rightarrow c_3 \rightarrow \cdots \rightarrow c_n \rightarrow d$$

Step 2 — Split the Grand Tour into sub-tours. Walk along the Grand Tour from the depot. Keep adding customers to the current sub-tour as long as the cumulative demand does not exceed the vehicle capacity Q . The moment adding the next customer would exceed Q , close the current sub-tour (return to depot), open a new one, and continue.

The Grand Tour Heuristic II

Worked Example: Splitting the Grand Tour

Grand Tour order: $d \rightarrow C_1 \rightarrow C_2 \rightarrow C_3 \rightarrow C_4 \rightarrow C_5 \rightarrow C_6 \rightarrow C_7 \rightarrow C_8 \rightarrow d$

Demands: $C_1=5$, $C_2=4$, $C_3=6$, $C_4=5$, $C_5=2$, $C_6=3$, $C_7=3$, $C_8=4$. Capacity $Q = 10$.

Step	Action	Cumulative demand
1	Start new route, add C_1	5
2	Add C_2 : $5 + 4 = 9 \leq 10$	9
3	Try C_3 : $9 + 6 = 15 > 10$ — exceed capacity! Close route: $d \rightarrow C_1 \rightarrow C_2 \rightarrow d$	—
4	New route, add C_3	6
5	Try C_4 : $6 + 5 = 11 > 10$ — exceed capacity! Close route: $d \rightarrow C_3 \rightarrow d$	—
6	New route, add C_4, C_5, C_6 : $5 + 2 + 3 = 10 \leq 10$	10
7	Try C_7 : $10 + 3 = 13 > 10$ — close route Close route: $d \rightarrow C_4 \rightarrow C_5 \rightarrow C_6 \rightarrow d$	—
8	New route, add C_7, C_8 : $3 + 4 = 7 \leq 10$ End of tour: close final route	7

The Clarke–Wright Savings Algorithm: Intuition I

The Clarke–Wright algorithm (Clarke and Wright, 1964) attacks the CVRP from the opposite end. Rather than starting with a TSP tour and splitting, it starts with the *worst* possible solution and iteratively *improves* it.

Worst-case starting point. Suppose we send a separate vehicle to every customer — n vehicles, each making a single out-and-back trip from the depot to one customer. This is obviously inefficient: every customer requires the full round-trip distance to the depot. The total distance is $2 \sum_{i=1}^n \text{dist}(d, c_i)$.

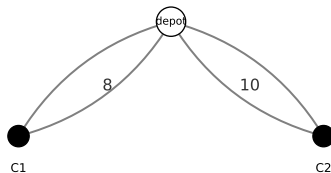
The saving idea. If two customers c_x and c_y can be served by a single vehicle (i.e., their combined demand is $\leq Q$), we can replace the two separate trips $d \rightarrow c_x \rightarrow d$ and $d \rightarrow c_y \rightarrow d$ with one merged trip $d \rightarrow c_x \rightarrow c_y \rightarrow d$. The **saving** is:

$$s(c_x, c_y) = \underbrace{\text{dist}(d, c_x) + \text{dist}(d, c_y)}_{\text{via depot}} - \underbrace{\text{dist}(c_x, c_y)}_{\text{direct link}}$$

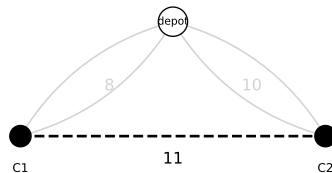
The saving $s(c_x, c_y)$ is positive whenever the detour via the depot is longer than going directly from c_x to c_y . A large saving means we gain a lot by combining these two customers on one route.

The Clarke–Wright Savings Algorithm: Intuition II

Via Depot: $d = 8 + 10 = 18$



Direct Link: $d' = 11 \rightarrow \text{Saving } s = 18 - 11 = 7$



Reading the figure. The left panel shows two separate routes: each van goes to its customer and comes straight back. The cost is $8 + 8 + 10 + 10 = 36$ distance units. The right panel merges the two customers: one van goes to C_1 , then directly to C_2 , then returns to the depot. The cost is $8 + 11 + 10 = 29$. The saving is $36 - 29 = 7$.

The Clarke–Wright Savings Algorithm: Intuition III

Savings Formula (Book Notation)

Let d denote the depot. For a pair of customers c_x and c_y :

- 1 Compute the **via-depot distance**:

$$d_{\text{via}} = \text{dist}(c_x, d) + \text{dist}(d, c_y)$$

- 2 Compute the **direct distance**:

$$d' = \text{dist}(c_x, c_y)$$

- 3 The **saving** is:

$$s = d_{\text{via}} - d' = [\text{dist}(c_x, d) + \text{dist}(d, c_y)] - \text{dist}(c_x, c_y)$$

The Clarke–Wright Savings Algorithm: Intuition IV

How to Read This

A positive saving means it is cheaper to go from c_x directly to c_y than to make two separate depot round-trips. The larger the saving, the more we gain by combining these two customers. If the saving is zero or negative, it is no cheaper (or more expensive) to combine them.

Numerical Example from the Book

$\text{dist}(C_1, \text{depot}) = 8$, $\text{dist}(\text{depot}, C_2) = 10$, $\text{dist}(C_1, C_2) = 11$.

$$d_{\text{via}} = 8 + 10 = 18, \quad d' = 11, \quad s = 18 - 11 = 7.$$

Since $s = 7 > 0$, merging C_1 and C_2 on one route saves 7 units of distance.

Clarke–Wright: The Full Algorithm I

The complete Clarke–Wright procedure is:

Algorithm 7: Clarke–Wright Savings Algorithm

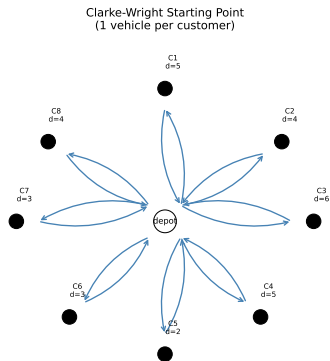
- ① **Initialise:** build one route per customer (the “flower” solution of Fig. 2.3 in the book).
- ② **Compute savings:** for every pair of customers (c_x, c_y) , compute $s = \text{dist}(c_x, \text{depot}) + \text{dist}(\text{depot}, c_y) - \text{dist}(c_x, c_y)$. Store all pairs with $s > 0$.
- ③ **Sort:** arrange the savings list in *descending* order, so that the most beneficial merge is considered first.
- ④ **Apply savings:** for each saving (s, c_x, c_y) in order:
 - ① Find the route r_x that contains c_x at its *end*.
 - ② Find the route r_y that contains c_y at its *start*.
 - ③ If $r_x \neq r_y$ and the combined demand $d(r_x) + d(r_y) \leq Q$: **merge** r_x and r_y into a single route.
- ⑤ **Return** the current solution (all remaining routes).

Clarke–Wright: The Full Algorithm II

Why must c_x be at the end and c_y at the start of their routes? Because the saving formula assumes the link goes $\dots \rightarrow c_x \rightarrow c_y \rightarrow \dots$, replacing the endings $\dots \rightarrow c_x \rightarrow d$ and $d \rightarrow c_y \rightarrow \dots$. If c_x were in the middle of a route, inserting a direct link to c_y would require reversing part of the route, which changes the distance calculation.

The algorithm considers savings from largest to smallest. This greedy ordering is not guaranteed to find the global optimum, but it ensures that the most beneficial merges are not blocked by suboptimal earlier decisions.

Clarke–Wright: The Full Algorithm III



The starting point: each customer has its own round-trip route (the “flower” pattern). The algorithm then merges pairs of petals into multi-customer routes.

Clarke–Wright: The Full Algorithm IV

Formal Pseudocode (Algorithm 7 from the book)

Procedure CLARKEWRIGHT()

savings $\leftarrow []$

solution $\leftarrow$ BUILDONEPERCUST()

for v_x in problem, **for** v_y in problem **do**

$d \leftarrow \text{dist}(v_x, \text{depot}) + \text{dist}(\text{depot}, v_y)$

$d' \leftarrow \text{dist}(v_x, v_y)$

$s \leftarrow d - d'$

if $s > 0$ **then** *savings*.add(Saving(s, v_x, v_y))

SORT(*savings*)

// descending by s

for saving s in *savings* **do**

$r_x \leftarrow \text{ROUTE}(s.x); r_y \leftarrow \text{ROUTE}(s.y)$

if $r_x.\text{last} == s.x$ **and** $r_y.\text{first} == s.y$ **then**

if $d(r_x) + d(r_y) \leq \text{cap}$ **then**

MERGE(r_x, r_y)

return *solution*

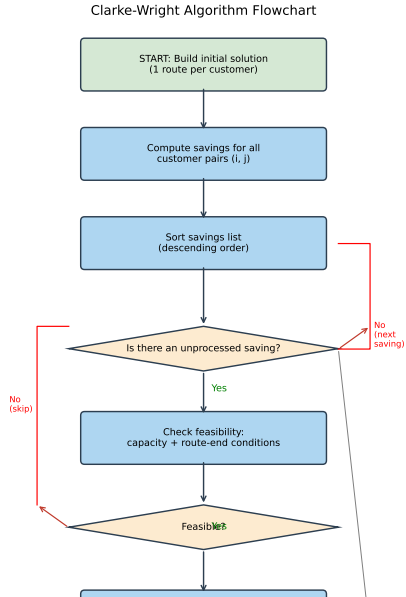
Clarke–Wright: The Full Algorithm V

The inner double loop (lines 3–8) runs over all pairs of customers, giving $O(n^2)$ savings to compute. Sorting is $O(n^2 \log n)$. Applying savings is $O(n^2)$. The overall time complexity is **$O(n^2 \log n)$** , which is efficient for hundreds of customers but becomes slow for thousands.

Clarke–Wright: Algorithm Flowchart I

The flowchart below shows the full Clarke–Wright procedure graphically. Each diamond represents a decision point; each rectangle is an action step. The red arrows show the “no” branches (loop back to the next saving or skip the current saving).

Clarke–Wright: Algorithm Flowchart II



Clarke–Wright: Step-by-Step Worked Example I

Let us trace through the Clarke–Wright algorithm on the 8-customer instance. We use simplified distances for illustration.

Initial solution: 8 routes, one per customer.

Sample savings (sorted largest-first):

Rank	Pair (c_x, c_y)	$d_{\text{via depot}}$	d' (direct)	Saving s
1	(C_1, C_2)	18	11	7
2	(C_4, C_5)	17	10	7
3	(C_3, C_4)	20	14	6
4	(C_6, C_7)	16	11	5
5	(C_2, C_3)	19	15	4

Clarke–Wright: Step-by-Step Worked Example II

Applying savings in order:

- ① **Saving 1** (C_1, C_2), $s = 7$: Both are at the end/start of their individual routes. Combined demand $5 + 4 = 9 \leq 10$. **Merge**: $d \rightarrow C_1 \rightarrow C_2 \rightarrow d$ (load = 9).
- ② **Saving 2** (C_4, C_5), $s = 7$: Both are at the end/start of their routes. Combined demand $5 + 2 = 7 \leq 10$. **Merge**: $d \rightarrow C_4 \rightarrow C_5 \rightarrow d$ (load = 7).
- ③ **Saving 3** (C_3, C_4), $s = 6$: C_3 is at the end of its route and C_4 is at the *start* of $d \rightarrow C_4 \rightarrow C_5 \rightarrow d$. Combined demand $6 + 7 = 13 > 10$. **Skip** (capacity violated).
- ④ **Saving 4** (C_6, C_7), $s = 5$: Both at end/start. Combined $3 + 3 = 6 \leq 10$. **Merge**: $d \rightarrow C_6 \rightarrow C_7 \rightarrow d$ (load = 6).
- ⑤ **(Continue)** ... Eventually C_5 might be merged with C_6 depending on route endpoint positions.

Clarke–Wright: Step-by-Step Worked Example III

Final solution may end up with 4 routes similar to: $\{C_1, C_2\}$, $\{C_3\}$, $\{C_4, C_5\}$, $\{C_6, C_7, C_8\}$ (exact result depends on all pairwise distances).

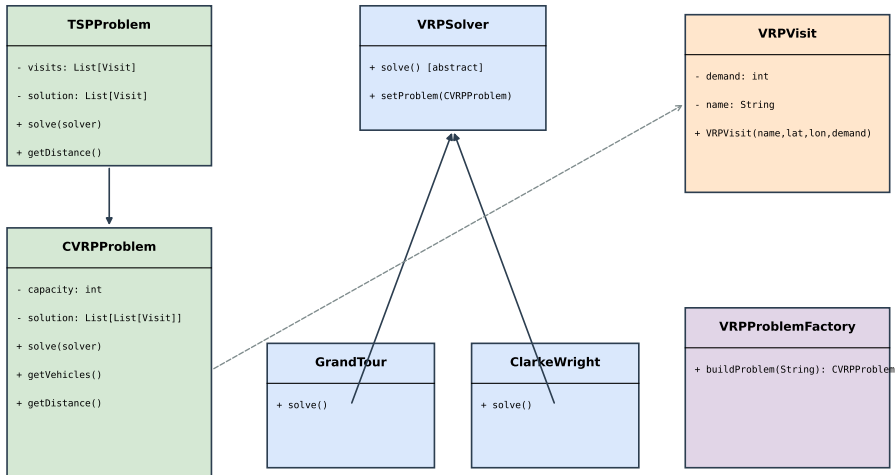
Key Insight

The Clarke–Wright algorithm is *greedy*: it always applies the largest available saving that is feasible. This does not guarantee an optimal solution, but in practice the greedy ordering works well because it prioritises the most geographically natural route merges first.

The book's implementation uses Java with an object-oriented class hierarchy that cleanly separates the *problem* from the *solver*. This design pattern — the Strategy pattern — is important because it allows different solvers to be swapped in without changing the problem class.

Software Architecture II

VRP Solver Architecture (simplified)



Python: Grand Tour Implementation

The following Python code implements the Grand Tour algorithm. It mirrors the Java `GrandTour.solve()` method from Listing 2.1 in the book, translated into idiomatic Python.

```
1  from dataclasses import dataclass
2  from typing import List
3  import math
4
5  @dataclass
6  class VRPVisit:
7      name: str
8      lat: float
9      lon: float
10     demand: int
11
12     def distance(self, other: 'VRPVisit') -> float:
13         return math.hypot(self.lat - other.lat, self.lon - other.lon)
14
15  class GrandTour:
16     """
17     Build a CVRP solution from a Grand Tour.
18     Assumes 'problem.solution' has already been set to a TSP tour
19     (flat list of VRPVisit) by a TSP solver such as NearestNeighbour.
20     """
21     def __init__(self, problem):
22         self.problem = problem      # CVRPPProblem object
23
24     def solve(self) -> List[List[VRPVisit]]:
```

Python: Clarke–Wright — Main Outline

The Clarke–Wright solver is built from three methods. Here is the main `solve()` method (corresponding to Listing 2.3 in the book):

```
1  from dataclasses import dataclass, field
2  from typing import List, Optional
3  import math, itertools
4
5  @dataclass(order=True)
6  class Saving:
7      saving: float          # sort key (descending after negation)
8      x: VRPVisit = field(compare=False)
9      y: VRPVisit = field(compare=False)
10
11  class ClarkeWright:
12      def __init__(self, problem):
13          self.problem = problem
14
15      def solve(self) -> List[List[VRPVisit]]:
16          # Step 1: Create initial solution (1 route per customer)
17          solution = [[v] for v in self.problem.visits]
18
19          # Step 2: Find all savings
20          savings = self._find_savings()
21
22          # Step 3: Sort savings largest-first
23          savings.sort(key=lambda s: -s.saving)
```

Python: Clarke–Wright — Finding Savings

The `_find_savings()` method iterates over every ordered pair of customers and computes the saving for each pair (Listing 2.4 in the book):

```
1     def _find_savings(self) -> List[Saving]:
2         savings = []
3         depot = self.problem.depot      # the depot VRPVisit object
4
5         # Consider every ordered pair (x, y) of customers
6         for x_idx, x in enumerate(self.problem.visits):
7             for y_idx, y in enumerate(self.problem.visits):
8                 if x_idx == y_idx:
9                     continue            # skip a customer paired with itself
10
11                 # Cost via depot: depot-to-x, x-to-depot, depot-to-y, y-to-depot
12                 # Saving = (x->depot + depot->y) - (x->y)
13                 cost_via_depot = (x.distance(depot)
14                                 + depot.distance(y))
15                 direct_dist    = x.distance(y)
16                 s              = cost_via_depot - direct_dist
17
18                 savings.append(Saving(saving=s, x=x, y=y))
19
20         return savings
```

Listing 3: Clarke–Wright: computing all pairwise savings

The time complexity of this method is $O(n^2)$ because it examines all $n(n-1)$ ordered pairs. For $n = 100$

Python: Clarke–Wright — Applying Savings

The `_apply_savings()` method tries to merge routes, checking the feasibility conditions (Listing 2.5 in the book):

```
1     def _apply_savings(self, solution, savings):
2         for s in savings:
3             route_x = route_y = None
4
5             # Find the routes containing s.x (as last element)
6             #                                     and s.y (as first element)
7             for route in solution:
8                 if route and route[-1] == s.x:    # s.x at END of route
9                     route_x = route
10                if route and route[0] == s.y:    # s.y at START of route
11                    route_y = route
12
13            # Merge only if both are found and belong to DIFFERENT routes
14            if route_x is not None and route_y is not None:
15                if route_x is not route_y:        # different routes
16                    combined_demand = (
17                        sum(v.demand for v in route_x)
18                        + sum(v.demand for v in route_y)
19                    )
20                    if combined_demand <= self.problem.capacity:
21                        # Merge: append route_y onto route_x
22                        route_x.extend(route_y)
23                        solution.remove(route_y)
```

Python: Running the CVRP Solvers

The following snippet shows how to set up a CVRP problem and run both solvers — mirroring the book's `main()` / Listing 2.2:

```
1  # Build a small test instance (8 customers, capacity 10)
2  visits = [
3      VRPVisit('C1', 0.0, 2.5, 5),
4      VRPVisit('C2', 2.2, 1.5, 4),
5      VRPVisit('C3', 2.8, 0.0, 6),
6      VRPVisit('C4', 1.8, -1.8, 5),
7      VRPVisit('C5', 0.0, -2.5, 2),
8      VRPVisit('C6', -1.8, -1.8, 3),
9      VRPVisit('C7', -2.8, 0.0, 3),
10     VRPVisit('C8', -2.2, 1.5, 4),
11 ]
12 depot = VRPVisit('depot', 0.0, 0.0, 0)
13
14 problem = CVRPPProblem(visits=visits, depot=depot, capacity=10)
15
16 # --- Grand Tour ---
17 nn_solver = NearestNeighbourSolver(problem)    # solve TSP first
18 nn_solver.solve()
19 gt_solver = GrandTour(problem)
20 gt_solution = gt_solver.solve()
21 print(f"Grand Tour:      distance={problem.get_distance():.1f}, "
22       f"vehicles={len(gt_solution)}")
23
24 # --- Clarke-Wright ---
```

Experimental Setup I

To evaluate the two heuristics, they are run on all 60 Augerat benchmark instances. For each instance, the following quantities are recorded:

- The **total distance** of the solution produced.
- The **number of vehicles** used.

These are then compared against the branch-and-cut optimal (or best-known) results published by Augerat (1995).

For the Grand Tour heuristic, the Nearest-Neighbour TSP solver is used to generate the initial tour (as in Listing 2.2 of the book).

Experimental Setup II

Performance Metric

The key metric is the **percentage excess above best known**:

$$\text{excess}(\%) = \frac{\text{heuristic distance} - \text{best-known distance}}{\text{best-known distance}} \times 100$$

A value of 0% means the heuristic matched the best-known solution exactly. A value of 25% means the heuristic's routes are 25% longer than optimal.

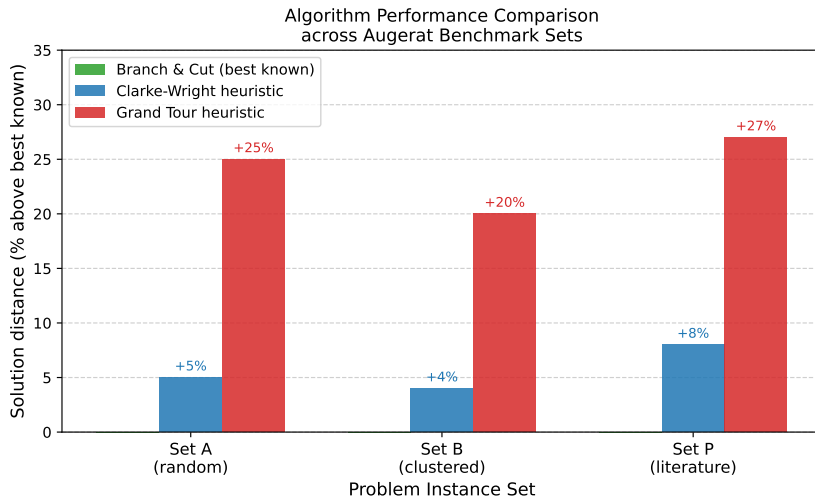
Results: Selected Instances from Table 2.1 (Set A) I

The table below shows a selection of Set A results. The “B&C” column gives the branch-and-cut reference; the other columns give the Grand Tour and Clarke–Wright results. Problem names have the form A-nX-kY, meaning Set A, $n = X$ customers, $k = Y$ vehicles in the best solution.

Problem	B&C dist	B&C veh	GT dist	GT veh	C-W dist	C-W veh
A-n32-k5	784	5	1023.9	5	810.8	5
A-n33-k5	661	5	777.8	5	712.0	5
A-n33-k6	742	6	880.5	6	776.3	7
A-n36-k5	799	5	975.6	5	834.4	5
A-n37-k5	669	5	968.2	5	707.3	5
A-n39-k5	822	5	1061.0	5	894.7	5
A-n45-k6	944	6	1160.7	6	1013.3	7
A-n53-k7	1010	7	1208.4	7	1097.6	7

Looking at A-n32-k5: the branch-and-cut optimum is 784; the Grand Tour produces 1023.9 (31% above optimum); Clarke–Wright produces 810.8 (only 3.4% above). Clarke–Wright is clearly the better heuristic for this instance.

Results: Selected Instances from Table 2.1 (Set A) II



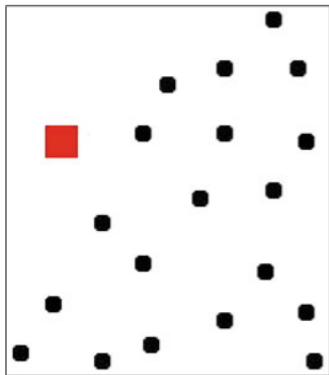
Results: Selected Instances from Table 2.1 (Set A) III

The bar chart aggregates results across all three Augerat sets. The key observation is that Clarke–Wright is substantially better than Grand Tour: it produces solutions only 5–8% longer than the best known, while Grand Tour is 20–27% longer. However, *both* heuristics are still noticeably worse than branch and cut — indicating that more sophisticated optimisation is needed.

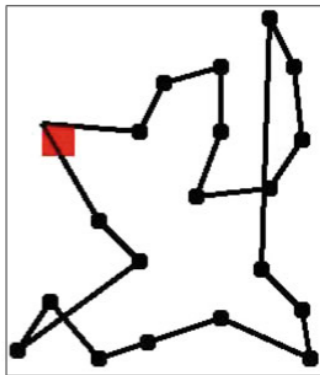
Results: Solution Visualisation I

The figure below (from the book's Fig. 2.5) shows the P-n20-k2 problem instance solved in three ways. Examining these visual solutions helps understand *why* the Grand Tour performs poorly.

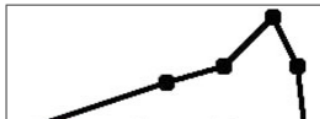
Results: Solution Visualisation II



(a) The P-n20-K2 problem instance.



(b) The initial TSP tour created using Nearest Neighbour.



Why Clarke–Wright Outperforms Grand Tour I

The performance gap between Clarke–Wright and Grand Tour has a clear structural explanation:

The Grand Tour heuristic is **sequential**: it first optimises for distance (via the TSP solver) *ignoring* the capacity constraint, then enforces the constraint by cutting the tour. The cuts are made mechanically at capacity boundaries without regard for geography — a cut might separate two customers who are close to each other.

Clarke–Wright is **capacity-aware from the start**: it begins with the capacity constraints fully respected (one vehicle per customer) and merges routes only when doing so is both feasible and beneficial. The savings criterion naturally groups geographically close customers together, because nearby customers have small direct distances and therefore large savings.

The Core Insight

The Clarke–Wright savings criterion is a proxy for geographic proximity: customers that are close to each other (small d') and far from the depot (large d_{via}) produce large savings. So the algorithm naturally groups nearby customers on the same route — exactly what a good delivery plan should do.

Conclusions and Limitations I

This chapter has introduced the CVRP and two classic heuristic solvers. The Grand Tour and Clarke–Wright algorithms both produce *feasible* solutions quickly — but how good are they in practice?

Grand Tour Assessment:

- Simple to implement: one loop over a sorted list.
- Very fast: $O(n)$ after the TSP tour is computed.
- Poor quality: 20–27% above optimal on average.
- The quality depends entirely on the TSP solver — if the Grand Tour has many long edges, the CVRP solution will also have long routes.
- Cannot group customers based on demand; the order is fixed by the TSP.

Clarke–Wright Assessment:

- More complex to implement, but still straightforward.
- Time complexity $O(n^2 \log n)$: slower than Grand Tour but efficient for hundreds of customers.
- Much better quality: 5–8% above optimal on average.
- Greedy strategy is not guaranteed to be optimal; it can get stuck in local optima.

Conclusions and Limitations II

The fundamental limitation of both heuristics is that they are *construction* methods: they build a solution in one pass and cannot escape from a poor choice made early on. Neither algorithm includes a mechanism to *improve* an existing solution after it has been built.

What Comes Next?

To achieve results comparable with branch-and-cut, we need a more powerful technique. The key insight is that we require two capabilities:

- 1 A way to **escape local optima** — to accept temporarily worse solutions in order to later find better ones.
- 2 A way to **leverage domain knowledge** — for example, by focusing on geographically natural customer groupings.

The subsequent chapters introduce nature-inspired metaheuristics (genetic algorithms, simulated annealing, ant colony optimisation) that address both of these needs and consistently outperform Clarke–Wright.

Summary of Chapter 2

The CVRP is NP-hard and requires either exponential-time exact methods (branch and cut) or fast heuristics. Clarke–Wright (1964) remains one of the best simple heuristics: easy to implement, $O(n^2 \log n)$, and typically 5–8% above optimal. It serves as the baseline that all more sophisticated solvers in this book aim to beat.

Chapter Summary: Key Formulas and Concepts I

CVRP Capacity Constraint

For every route r_j :

$$\sum_{i \in r_j} \delta_i \leq Q$$

Each δ_i (delta) is the demand of customer i ; Q is the vehicle capacity. No route can carry more than Q units.

Minimum Vehicles Lower Bound

$$k_{\min} = \left\lceil \frac{\sum_{i=1}^n \delta_i}{Q} \right\rceil$$

The total demand divided by capacity, rounded up. This is a theoretical minimum; the actual solution may require more vehicles.

Clarke–Wright Saving

$$s(c_x, c_y) = \text{dist}(c_x, d) + \text{dist}(d, c_y) - \text{dist}(c_x, c_y)$$

A positive saving means it is cheaper to link c_x directly to c_y than to route both via the depot separately.

Chapter Summary: Key Formulas and Concepts III

Algorithm Complexity Summary

Algorithm	Time complexity	Solution quality
Grand Tour	$O(n^2)$ (with NN TSP)	20–27% above optimum
Clarke–Wright	$O(n^2 \log n)$	5–8% above optimum
Branch and cut	Exponential (worst case)	Optimal

Augerat Benchmark Sets

- **Set A:** 30 instances, random customer placement, $n = 31$ –80 customers.
- **Set B:** 20 instances, clustered customer placement, $n = 31$ –78.
- **Set P:** 24 instances, literature-based instances, $n = 16$ –101.

All instances have known optimal or best-known solutions. The total collection of 60 problems provides a comprehensive evaluation platform.

Bottom Line

Clarke–Wright is fast, simple, and produces solutions that are practically useful (within 8% of optimal on typical instances). It is the standard baseline against which all more advanced CVRP solvers are measured. Nature-inspired methods introduced in later chapters consistently achieve 2–4% above optimal — roughly halving the error of Clarke–Wright.

References I

- Augerat, P. (1995). *Approche polyédrale du problème de tournées de véhicules* [Polyhedral approach of the vehicle routing problem]. PhD thesis, Grenoble Institute of Technology, France.
- Augerat, P. (2014a,b,c). VRP-REP: Augerat 1995 Sets A, B, P.
<http://www.vrp-rep.org/datasets/>
- Clarke, G., and Wright, J.W. (1964). Scheduling of vehicles from a central depot to a number of delivery points. *Operations Research*, 12(4), 568–581.
- Dantzig, G.B., and Ramser, J.H. (1959). The truck dispatching problem. *Management Science*, 6(1), 80–91.
- Gendreau, M., et al. (2008). Metaheuristics for the vehicle routing problem and its extensions: a categorized bibliography. In *The Vehicle Routing Problem: Latest Advances and New Challenges*. Springer.
- Laporte, G.T. (2013). Vehicle routing: historical perspective and recent contributions. *EURO Journal on Transportation and Logistics*, 2(1–2).
- Solomon, M.M. (1987). Algorithms for the vehicle routing and scheduling problems with time window constraints. *Operations Research*, 35(2), 254–265.